

# Topic - Computer Systems Architecture

Series 13, 14, 15

November 2024

Last update: 17.12.2024

## content

|       |                                                            |    |
|-------|------------------------------------------------------------|----|
| 1     | Administrative details                                     | 2  |
| 1.1   | Deadlines. ....                                            | 2  |
| 1.2   | Reminder of the score on the topic .                       | 2  |
| 1.3   | Transmission .                                             | 2  |
| 1.4   | What will be transmitted .                                 | 2  |
| 1.5   | How the evaluation will be done .                          | 2  |
| 1.6   | Other observations .                                       | 3  |
| 2     | Formulation of the theme                                   | 4  |
| 2.1   | Presentation of requirements . ....                        | 4  |
| 2.1.1 | Implementing the one-dimensional memory system . ....      | 4  |
| 2.1.2 | Implementation of the two-dimensional memory system . .... | 5  |
| 2.2   | Example of operation .                                     | 5  |
| 2.3   | Example for the one-dimensional case .                     | 5  |
| 2.4   | Example for the two-dimensional case .                     | 6  |
| 3     | Requirements                                               | 9  |
| 3.1   | Requirement 0x00 - one-dimensional case . ....             | 9  |
| 3.2   | Requirement 0x01 - two-dimensional case .                  | 10 |

# 1 Administrative details

## 1.1 Deadline

You can either send the solutions no later than January 5, 2025, 11:59 p.m.

## 1.2 Reminder of the score on the topic

The assignment is worth 25% of the grade for this lab (according to Course 0x00), and a grade of 5 is required to pass.

## 1.3 Transmission

You will submit the solutions in the following forms, depending on the group (or depending on the series, or depending on the specialization):

- groups 131 and 152: <https://forms.gle/sv5YdukZEcDHDS6HA>
- groups 132 and 151: <https://forms.gle/duxvGeAKnkiPqyXF8>
- groups 133 and 134: <https://forms.gle/7h9bwHK9TstX7fYp8>
- series 14: <https://forms.gle/nXNYbVcNqJpAmR3Q6>
- mathematics-computer science: <https://forms.gle/CPDa5uyyikctj4Uy7>
- arrears: <https://forms.gle/gE9ufvCuLz7zZAC89>

## 1.4 What will be transmitted

They will be loaded into the form. two sources (one for each requirement) with the names group - last name first name 0.s, respectively group name first name 1.s. If you have multiple first/last names, you will load named sources, for example, 172.GeorgescuXulescu IonVasile 0.s. It is important to upload sources with the correct name, because testing will be automatic.

## 1.5 How the evaluation will be done

There are two steps to obtain note taking:

- all sources are checked for plagiarism. If plagiarism is detected, a notification is automatically made to *Ethics Committee of the University of Bucharest*,
- Sources that have passed the anti-plagiarism check will be automatically tested.

Important! Students who have different configurations than those we work on in the laboratory must specify this in the form in which they submit the assignment, in order to be able to perform the evaluation and not receive a default 0.

## 1.6 Other observations

1. We don't forbid you from discussing ideas among yourselves, but be careful, because there is an important difference between giving an idea and giving the code directly.
2. Do not use automatic converters from C/C++/other languages to x86, we have used them too and we easily recognize code that is not written by you.

## 2 Formulating the theme

To implement this theme, we assume that you are part of the development team of a *operating system* minimal - the operating system is a software product that deals with the management and coordination of the activities of a computing system, with a role in mediating the access of application programs to the machine's resources. The task that will be left is to implement a storage device management component (*Hard Drive*OR*SSD*), and because the project is just beginning, we have many assumptions that simplify the development of this product.

### 2.1 Presentation of requirements

Two operating modes are taken into account, a case in which the memory is linear, one-dimensional, and a case in which the memory is two-dimensional.

#### 2.1.1 Implementation of the one-dimensional memory system

In this one-dimensional case, the way you are asked to operate the storage device is as follows:

- the device's storage capacity is given and fixed at 8MB;
- the device's storage capacity is divided into blocks of 8kB each;
- A block can store content from a single file;
- a file needs at least two blocks for storage;
- it is assumed that a stored file is stored contiguously;
- If a file cannot be stored contiguously then writing it to the device is not possible.

The operating system does not have a directory and file structure, it just has to store files. In this sense, each file is identified by a unique descriptor - ID (a natural number between 1 and 255); thus, our system can store a maximum of 255 different files.

We are interested in the storage device management module being able to perform the following operations:

- given a descriptor (file ID), return the block range (start, end) where the file is stored;
- given a file descriptor and its size in kB, return the range of blocks where the file can be stored. The first free range will be returned, in the left-to-right traversal. If storage is not possible, the range (0, 0);
- given a descriptor, to delete the respective file (i.e. to free the blocks where the file content was saved); we consider *CLEAR* the operation by which blocks receive the value as descriptor 0;
- defragmentation operation: reorder/recalculate the blocks in which files are stored, so that they are stored compactly (i.e. starting with block 0 and using all consecutive blocks, without gaps).

### 2.1.2 Implementation of the two-dimensional memory system

After you have completed the implementation in the one-dimensional case, you notice that the space you have fills up quite quickly, so you think about a natural extension, in two dimensions this time, with a size of 8MB in both directions. Thus, we now assume that the storage device is two-dimensional, that is, we have an array of blocks, and a contiguous section is considered in lines.

- given a file descriptor, return the block range ((startX, startY), (endX, endY)) where the file is stored;
- given a file descriptor and its size in kB, return the block IDs where the file can be stored. The first interval in which the file can be positioned will be returned. If it cannot be stored then the interval ((0,0), (0,0))
- given a file descriptor, delete the respective file (i.e. free the blocks where the file content was saved); just as in the one-dimensional case, we consider *CLEAR* the operation by which the blocks receive the value 0 as descriptor;
- the two-dimensional defragmentation operation; reorder the blocks in which the files are stored, so that they are stored compactly in the array (we move the gaps to the bottom right in the array).

## 2.2 Example of operation

For the implementation of the theme, you will also consider a simplified representation of the memory size, just to be able to demonstrate that the file system management works. Thus, you will reduce the 8kB of a block to 8b (1 Byte), and you will keep the same conversion for the memory size.

## 2.3 Example for the one-dimensional case

Initially, your memory is 0 in all blocks:

0, 0, 0, 0, 0, 0, ..., 0, 0, 0

ADD If a file with descriptor 5 and a size of 20kB is added, the change will be as follows:

- descriptor 5 tells us that we will modify the blocks with the value 5;
- the size of 20kB is reduced in representation to 20b, which means that we need 3B (the upper approximation) for representation - so we need 3 blocks;
- we will allocate these blocks in order.

5, 5, 5, 0, 0, 0, ..., 0, 0, 0

ADD Suppose we add a file with descriptor 143 and a size of 14kB. Then, for it we need two blocks, and the memory will now become:

5, 5, 5, 143, 143, 0, ..., 0, 0, 0

GET If we want to query the operating system and tell us where to find the file with descriptor 143, it will return the range (3, 4), and if we ask for the file with descriptor 10, it will return the range (0, 0), because this file does not exist.

DELETE Let's assume we delete the file with descriptor 5. In this case, the resulting memory looks like this:

0, 0, 0, 143, 143, 0, ..., 0, 0, 0

ADD Let's assume we add another file, this time of 73kB, with descriptor 25. For its storage we consider 73b, which means as a higher approximation 10B, so 10 blocks. They are allocated, in order, after the last descriptor we filled, so after descriptor 143. The memory will look like this:

0, 0, 0, 143, 143, 25, 25, 25, 25, 25, 25, 25, 25, 25, 0, ..., 0, 0, 0

ADD Suppose we add a file with descriptor 251 and size 12kB. In the representation we have 12b, so 2B, corresponding to two blocks. Since we already have three free blocks at the beginning, the file will fit in the first two of them.

251, 251, 0, 143, 143, 25, 25, 25, 25, 25, 25, 25, 25, 25, 0, ..., 0, 0, 0

DELETE Let's assume we delete the file with descriptor 25. In this case, the memory will look like this:

251, 251, 0, 143, 143, 0, 0, 0, 0, 0, 0, 0, 0, 0, ..., 0, 0, 0

DEFRAGMENTATION Now, suppose we want to defragment the memory, so that the storage is compact again and allow new files to be stored at lower memory addresses, not just after the file with descriptor 251. After applying this operation, the files with descriptors 143 and 255 will be at lower memory addresses, stored consecutively:

251, 251, 143, 143, 0, 0, 0, ..., 0, 0, 0

ADD Now, if we assume adding a file with descriptor 4, which occupies 22kB, the memory will be modified as follows:

251, 251, 143, 143, 4, 4, 4, 0, 0, 0, ..., 0, 0, 0

Observation. If we now delete the file with descriptor 4, and add a new one, with descriptor 29, since the file with descriptor 4 was the last allocated, the one with 29 will be placed immediately after 143.

## 2.4 Example for the two-dimensional case

Initially, your memory is 0 in all blocks:

0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0,  
0, ..., 0, 0, 0, 0, ...

0, 0, 0, 0, 0, ..., 0, 0, 0, 0, 0, 0,  
0, ..., 0, 0, 0

For simplicity, we will consider a complete, but smaller, matrix to illustrate the operations (we will not consider 8MB x 8MB memory in this example, but you should consider it as such in the lesson). In the following example, we will consider 64kB by 64kB memory.

```
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

ADDWe add a file with descriptor 5, respectively a size of 50kB. (occupies 7 blocks)

```
5, 5, 5, 5, 5, 5, 5, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

ADDWe add a file with descriptor 2, respectively a size of 52kB. (it also occupies 7 blocks)

```
5, 5, 5, 5, 5, 5, 5, 0
2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0
```

ADDWe proceed exactly as follows and add the files with descriptors 1, 3, 4, 6, 7, 8, with sizes that occupy 2, 4, 3, 5, 2, and 8 blocks, respectively.

```
5, 5, 5, 5, 5, 5, 5, 0
2, 2, 2, 2, 2, 2, 2, 0
1, 1, 3, 3, 3, 3, 7, 7
4, 4, 4, 6, 6, 6, 6, 6
8, 8, 8, 8, 8, 8, 8, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

Observation. The file with descriptor 7 was placed immediately after the area for the file with descriptor 3, because there were already two blocks available.

GETIf we want to query the operating system and tell us where to find the file with descriptor 3, it returns the range ((2, 2), (2, 5)).

DELETESuppose we want to delete files with descriptors 1, 4, 8. The result in memory is:

```
5, 5, 5, 5, 5, 5, 5, 0
2, 2, 2, 2, 2, 2, 2, 0 0, 0, 3,
3, 3, 3, 7, 7 0, 0, 0, 6, 6, 6,
6, 6 0, 0, 0, 0, 0, 0, 0 0,
0, 0, 0, 0, 0, 0 0, 0, 0, 0,
0, 0, 0, 0 0, 0, 0, 0, 0, 0,
0
```

DEFAGMENTATIONNow, let's say we want to defragment the memory so that the storage is compact again. So, we want the 0s to end up in the bottom right corner.

```
5, 5, 5, 5, 5, 5, 5, 0
2, 2, 2, 2, 2, 2, 2, 0
3, 3, 3, 3, 7, 7, 0, 0
6, 6, 6, 6, 6, 0, 0, 0 0, 0, 0,
0, 0, 0, 0, 0 0, 0, 0, 0, 0,
0, 0 0, 0, 0, 0, 0, 0, 0 0,
0, 0, 0, 0, 0, 0, 0
```

Observation.Files are stored in lines, and in defragmentation we preserve the order of the files, trying to fill in the gaps where possible.



### 3 Requirements

Observation! In the topic we consider 1 MB = 1024 KB, respectively 1 kB = 1024 B.

Within this topic, you have two requirements: a requirement for the one-dimensional approach, for which you will receive 50p, and a requirement for the two-dimensional part, for which you will receive 40p. Important! NO enter the input manually every time you retest the program! These are long inputs that will cost you time. Create a file, for example `input.txt`, where you write the desired input, and after you have an executable, for example `task00`, which you would normally run with `./task00`, run the command `./task00 < input.txt`. Thus, the content of the file will be redirected to STDIN, just as if you had entered the values manually. Use this information to test your inputs by creating your own files. `input0.txt`, `input1.txt` etc., and testing them with `./task00 < input0.txt`, `./task00 < input1.txt` And so on

Important! All character strings (used for display) that you define in the section `date` will have, at the end, the character `' \n'`!

#### 3.1 Requirement 0x00 - one-dimensional case

This requirement will allow you to obtain 50p, of which:

- You get 10p just for implementing the ADD functionality;
- You get 10p if you have implemented ADD and GET;
- You get 10p if you have implemented ADD and DELETE;
- You get 5p if you have implemented ADD, DELETE, GET;
- You get 15p if you have implemented ADD, DELETE, GET, DEFRAGMENTATION.

(you get 10p for ADD, 20p for implementing GET or DELETE, 35p if there are both ADD, GET, and DELETE, respectively 50p if there are all of them)

In implementing the first requirement, you have the following input:

- the first line of input contains the number `O` of operations to be performed;
- starting with the second line, you will receive each operation, the coding of each being:
  - 1 - ADD
  - 2 - GET
  - 3 - DELETE
  - 4 - DEFRAGMENTATION
- if the operation is ADD, then:
  - the next line contains a number `N`, representing the number of files to be added;
  - on the next `2N` lines you will receive, successively, the file descriptor, respectively the size in kB of the respective file;

- you will return intervals (represented by the block IDs) into which the given files were inserted, in the form "%d: (%d, %d)\n", where the first integer is the descriptor, the second is the left end, the third is the right end. Intervals are considered intervals closed;
- if a file cannot be added, it is displayed fd: (0, 0) (where fd is the descriptor you were supposed to add)
- if the operation is GET, then you will only receive a file descriptor on the next line and the range in which the file is found will be returned, respectively the range (0, 0) if the file does not exist; The display format must be in the form "(%d, %d)\n"
- if the operation is DELETE, then you will only receive a file descriptor on the next line, and the updated memory will be returned; if the file to be deleted exists, then it will no longer be contained in memory (the blocks will be replaced with 0), otherwise the memory will be displayed without any changes;
- if the operation is DEFRAGMENTATION, the defragmented memory will be displayed, in the same format as after performing the operation ADD, according to the explanations in the previous section (the continuity of the blocks that store the files will be ensured).

| Input                                                                       | Input explanation                                                                                                                                                                                                                                                                        | Output                                                                         | Output explanation             |
|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------|
| 4<br>1<br>5<br>1<br>124<br>4<br>350<br>121<br>75<br>254<br>1024<br>70<br>30 | 4 operations are performed<br>ADD is performed<br>number of files<br>first file descriptor<br>size in kB<br>the second file descriptor<br>size in kB<br>the third file descriptor<br>size in kB<br>the fourth file descriptor<br>size in kB<br>the descriptor of the fifth<br>size in kB | 1: (0, 15)<br>4: (16, 59)<br>121: (60, 69)<br>254: (70, 197)<br>70: (198, 201) | output first operation         |
| 2<br>121                                                                    | GET is performed<br>the file descriptor we are doing a GET on                                                                                                                                                                                                                            | (60, 69)                                                                       | output of the second operation |
| 3<br>4                                                                      | DELETE is performed<br>the file descriptor on which we are doing the DELETE                                                                                                                                                                                                              | 1: (0, 15)<br>121: (60, 69)<br>254: (70, 197)<br>70: (198, 201)                | output of the third operation  |
| 4                                                                           | DEFRAGMENTATION is performed                                                                                                                                                                                                                                                             | 1: (0, 15)<br>121: (16, 25)<br>254: (26, 153)<br>70: (154, 157)                | output of the fourth operation |

### 3.2 Requirement 0x01 - two-dimensional case

This requirement will allow you to obtain 50p, of which

- 10p is obtained only from implementing the ADD method;

- another 10p is obtained only if ADD, GET, DELETE are implemented;
- another 10p is obtained only if ADD, GET, DELETE, DEFRAGMENTATION are implemented;
- 20p is obtained if the theme is fully implemented, but the files used are existing files on disk.

In implementing this requirement, you have the following input:

- the first line of input contains the number O of operations to be performed;
- starting with the second line, you will receive each operation, the coding of each being:
  - 1 - ADD
  - 2 - GET
  - 3 - DELETE
  - 4 - DEFRAGMENTATION
- if the operation is ADD, then:
  - the next line contains a number N, representing the number of files to be added;
  - on the next 2N lines you will receive, successively, the file descriptor, respectively the size in kB of the respective file;
  - you will return intervals (represented by the block IDs) into which the given files were inserted, in the form "%d: ((%d, %d), (%d, %d))\n", where the first integer is the descriptor, the second is the left end, the third is the right end. Intervals are considered intervals closed;
  - if a file cannot be added, it is displayed as fd: ((0, 0), (0, 0)) (where fd is the descriptor you were supposed to add)
- if the operation is GET, then you will only receive a file descriptor on the next line and the range in which the file is found will be returned, namely the range ((0, 0), (0, 0)) if the file does not exist; The display format must be in the form "((%d, %d), (%d, %d))\n"
- if the operation is DELETE, then you will only receive a file descriptor on the next line, and the updated memory will be returned; if the file to be deleted exists, then it will no longer be contained in memory (the blocks will be replaced with 0), otherwise the memory will be displayed without any changes;
- if the operation is DEFRAGMENTATION, the defragmented memory will be displayed, in the same format as after performing the operation ADD, according to the explanations in the previous section (the continuity of the blocks that store the files will be ensured).

In addition, operation 5 - CONCRETE is also considered, in which case a *file path* absolutely, and the already implemented ADD operation is used, so that the bidirectional memory is filled according to the files already existing in *path* The specified file. To solve this task, from the respective path, for each file, the descriptor (modulo 255 + 1), respectively the size in kB of the file, will be determined.

The CONCRETE operation works as follows:

- in input we receive 5 and one *path* absolutely to the folder where we have the files;
- files are text files, you can think of them as being named in the form file0.txt, file1.txt, ..., fileN.txt;
- Your program must:
  - determine a *file descriptor* for each file; you will do this through a system call `open`. Be careful that `open` returns a real descriptor, you need to convert that descriptor into a measure between 1 and 255 (you can use as `int fd = (fds % 255) + 1`;
  - determines the size in kB of the open file.
- since for each file we will know a pair of shapes (file descriptor, size in kB), the CONCRETE operation will be treated exactly like an ADD;
- as you find the files, display this information on the screen, the descriptor calculated (the modulo  $255 + 1$ ), respectively the size in kB, along with the range where it can be filled in memory;
- if a calculated descriptor is repeated, display it (also in the form of writing the descriptor, respectively the size, to STDOUT), display `fd: ((0, 0), (0, 0))` but DO NOT add it to your two-dimensional memory.

For example, in the C language, you would write the following:

```
targetfds=open(filepath, O_RDONLY);
targetfd=(sdfs% 255)+ 1;

structurestatefileStat;
fstat(fds,&fileStat);
Longasize=fileStat.st_size/ 1024; close
(fds);
```